Benchmark Evaluation: TPC-DI and MIMIC-IV → OMOP

Scope, known findings, and work plan

1 Background and scope

1.1 The research problem

Companies frequently have to move data from one database to another — for example, when two companies merge, or when an old system is replaced. The old database and the new one usually store the same information in *different shapes*. Someone has to write a program that reads the old data and writes it into the new shape. That program is called a **transformation** (or **ETL**, see glossary).

Our research concerns generating such programs automatically and verifying their correctness. That requires examples to evaluate against; this document covers sourcing and preparing them.

1.2 The four things we need

For each example we need all four of these. If any one is missing, the example is not usable.

Symbol	Name	What it is
S	Source schema	The *structure* of the old database — which tables exist, which columns they have, what type each column is. Written as SQL `CREATE TABLE` statements.
D	Source data	The actual rows sitting in the old database.
T	Target schema	The structure of the new database. Also `CREATE TABLE` statements.
—	Transformation	The program (usually SQL) that reads D and fills the new database. This is the "correct answer" we compare against.

Two candidate sources are under evaluation:

1. **TPC-DI** — a financial-trading benchmark. Section 2.
2. **MIMIC-IV → OMOP** — a hospital-data conversion. Section 3.

1.3 Terminology

Term	Meaning
DDL	"Data Definition Language". The SQL commands that create structure, i.e. `CREATE TABLE`. As opposed to commands that read or write data.
Surrogate key	An identifier *invented* by the transformation, which does not exist in the source. Usually just 1, 2, 3, ... Important: two correct transformations may number them differently, and *both are right*.
ETL	"Extract, Transform, Load". The general name for a program that reads data from one place, changes its shape, and writes it somewhere else.

Term	Meaning
Star schema	A common warehouse design. A few large "fact" tables recording events (trades, sales), surrounded by smaller "dimension" tables describing things (customers, dates). Named because a diagram looks like a star.
SCD Type 2	"Slowly Changing Dimension, Type 2". A way of keeping history. When a customer moves house, you do *not* overwrite the old address. You mark the old row as finished (`EndDate`) and add a new row marked current (`IsCurrent = true`). One real customer becomes several rows.
DuckDB	A database that lives in one file on your laptop. No server to install and run — one `pip install` and you are done. We use it because it is the least setup.
dbt	A tool where each output table is defined by one SQL file. Useful to us because it means the transformation for each table is a separate, readable file rather than one huge script.
Scale factor (SF)	A dial on a data generator saying how much data to make. SF=10 for TPC-DI is roughly 1 GB. Always start at the smallest.

1.4 Environment

The toolchain assumes Linux. On Windows, use WSL.

```
# 1. Open PowerShell AS ADMINISTRATOR, run this, then restart the computer:
wsl --install -d Ubuntu-22.04

# 2. After restart, open "Ubuntu" from the Start menu and set a username/password.

# 3. Inside Ubuntu, run:
sudo apt update
sudo apt install -y git python3 python3-venv python3-pip unzip

# 4. Verify:
git --version
python3 --version
```

VS Code with the WSL extension (https://code.visualstudio.com/) is convenient for reading the SQL files.

All work goes under `~/benchmark/`:

```
mkdir -p ~/benchmark
cd ~/benchmark
```

1.5 Record of work

Keep `~/benchmark/FINDINGS.md` updated per task: what was run, what happened, and the verdict. Paste error messages verbatim. This file is the primary deliverable.

2 Benchmark 1: TPC-DI

2.1 What TPC-DI is

TPC is an organisation that publishes standard tests for database systems. TPC-DI is their test for data integration. It describes an imaginary stockbroking company:

- **The source** is a messy pile of files — customer records, trade records, financial statements — in several different formats. This is our D.
- **The target** is a clean data warehouse with 20 tables. This is our T.
- **The rules** for getting from one to the other are written in a specification document, in English prose. Note: *prose, not code*. TPC gives the requirements; other people have written the actual programs.

2.2 What we already know (verified)

Finding	Detail
T is ready	The target schema has been extracted, converted to portable SQL, and tested. It loads cleanly on DuckDB: **20 tables, 30 foreign keys, 241 columns**. File: `tpcdi_target_schema.sql` (supplied with this document).
The work is non-trivial	Of the 241 target columns, **97 (40%) need more than a straight copy** from the source. Breakdown: 30 need surrogate-key lookups, 15 are SCD Type 2 history bookkeeping, 11 are derived calendar labels, 11 need lookup joins, 4 need window functions over history, and the rest are conditionals and arithmetic.
Five tables keep history	`DimCustomer`, `DimCompany`, `DimSecurity`, `DimBroker`, `DimAccount` all use SCD Type 2.
Four tables are trivial	`Industry`, `StatusType`, `TaxRate`, `TradeType` are pure copies. Useful as an easy baseline.

2.3 What is missing

	Missing piece	Why
S	The source schema	TPC-DI's source is *files*, not a database. Somebody has to write `CREATE TABLE` statements for each file.
D	The source data	Must be generated by a program.
—	The transformation	TPC publishes rules in English, not code. Third parties have written implementations.

2.4 Repositories to investigate

#	Repository	Why it matters
R1	abroniewski/ TPC-DI-MS-SQL-Benchmark	**Investigate this first.** Appears to contain both a source-schema script and a single SQL file with all the transformations. If true, it solves two of our three missing pieces.
R2	haydarai/tpcdi-kit	Appears to be a data generator. If it works without registering with TPC, it solves the third piece.
R3	rlsalcido24/dbtpcdi	A dbt implementation targeting several database systems, so its SQL should be more portable.
R4	shannon-barrow/ databricks-tpc-di	The largest and best-maintained implementation, but built for the Databricks cloud platform. May not run on a laptop.
R5	detobel36/tpc-di	Where our target schema came from. Its transformations are in a format we cannot read easily (SSIS), so only the schema file is useful.

The "Why it matters" column records expectations drawn from repository descriptions, not verified facts — these files have not been opened. Confirming or refuting each expectation is the purpose of tasks A1–A3.

2.5 TPC-DI tasks

Task A1 — Clone the repositories and look inside *2 hours*

Goal. Establish the contents of each repository.

Steps. From `~/benchmark`:

```
cd ~/benchmark
mkdir -p tpcdi && cd tpcdi
git clone https://github.com/abroniewski/TPC-DI-MS-SQL-Benchmark.git
git clone https://github.com/haydarai/tpcdi-kit.git
git clone https://github.com/rlsalcido24/dbtpcdi.git
```

Now list what is inside each. From `~/benchmark/tpcdi`:

```
find TPC-DI-MS-SQL-Benchmark -type f | head -50
find tpcdi-kit -type f | head -50
find dbtpcdi -type f -name "*.sql" | head -50
```

Deliverable. In `FINDINGS.md`, paste the file listings. Then answer:

1. Does `TPC-DI-MS-SQL-Benchmark` contain a file named `CreateDBTableSchema.sql`? What is its full path?
2. Does it contain `historical_load.sql`? What is its full path?
3. How many lines does each have? Use `wc -l <filename>`.
4. Does `tpcdi-kit` contain a README explaining how to generate data? Does the README mention needing to register or download anything from tpc.org?

Task A2 — Read the source schema file *2 hours*

Goal. Determine whether `CreateDBTableSchema.sql` supplies S.

Steps. Open the file in an editor.

For each `CREATE TABLE` statement you find, record: the table name, the number of columns, and the first three column names with their types.

Example of what a good answer looks like:

```
CREATE TABLE source.Customer (
    CustomerID INT,
    TaxID CHAR(20),
    Status CHAR(10),
    ...
);
```

Table	Columns	First three columns
`source.Customer`	23	`CustomerID INT`, `TaxID CHAR(20)`, `Status CHAR(10)`

Deliverable. A table in `FINDINGS.md` listing every source table, and a one-line verdict: *"This file does / does not give us S."*

Task A3 — Classify the transformations *1 day. This is the most important task.*

Goal. Characterise the transformation classes TPC-DI requires. This determines whether we adopt the benchmark.

Rationale. If most target columns are plain copies, the benchmark will not discriminate between methods. The distribution across transformation classes is the deciding measure.

The categories. Every target column falls into exactly one of these:

Category	**What it means**	**Example**
copy	Value taken unchanged from one source column	`CustomerID`
rename	Same value, different column name	`cust_id` → `CustomerID`
retype	Value converted to a different type	text `'2024-01-05'` → a date
concat	Two or more source columns glued together	`first_name || ' ' || last_name`
conditional	Value depends on a test	`CASE WHEN rating < 'BBB' THEN 1 ELSE 0 END`
arithmetic	Value computed by calculation	`revenue / earnings`
lookup join	Value fetched from another table	status code `'ACTV'` → `'Active'`
aggregate	Value summarises many rows	`SUM`, `COUNT`, `MAX`
window	Value depends on a range of other rows	highest price in the last 52 weeks
key generation	A number invented by the transformation	`ROW_NUMBER()`
SCD history	The `IsCurrent` / `EffectiveDate` / `EndDate` columns	—
constant	A fixed value, not from the source	`BatchID = 1`

Steps.

1. Open `historical_load.sql` in VS Code.

2. Find the part that fills `DimCustomer`. Search for `INSERT INTO DimCustomer` or `DimCustomer AS`.
3. For *every column* it produces, decide which category it belongs to.
4. Do the same for `DimAccount` and `FactMarketHistory`.

Deliverable. A spreadsheet, `tpcdi_transformations.csv`, with these columns:

target_table	target_column	category	source_columns	sql_snippet
DimCustomer	CustomerID	copy	Customer.CustomerID	`c.CustomerID`
DimCustomer	Status	lookup join	Customer.Status	`JOIN StatusType st ON ...`
DimCustomer	IsCurrent	SCD history	—	`CASE WHEN ... THEN 1`
DimCustomer	MarketingNameplate	conditional	NetWorth, Income, ...	`CASE WHEN ...`

Plus a count of how many columns fell into each category.

> **Cross-check.** Analysis of the target schema alone gave approximately 60% plain copies and 40% more complex. A materially different count from the actual SQL means one of the two analyses is wrong; report the discrepancy.

Task A4 — Find out how to get the data *half a day*

Goal. Determine whether we can generate D without registering with TPC.

Context. The official generator, `DIGen.jar`, requires registration and a licence agreement with TPC. Several third-party replacements exist; the question is whether any of them works standalone.

Steps.

1. Open the README of `tpcdi-kit` (R2). Does it include the generator itself, or does it tell you to download `DIGen.jar` from TPC first?
2. If it includes a generator, follow its instructions and try to produce data at the smallest scale factor.
3. Report exactly what files were produced, their names, and their sizes. Use:

```
ls -lh <output-folder>
```

4. Open one of the produced files with `head -5 <filename>` (shows the first 5 lines) and paste those lines into `FINDINGS.md`.

If registration proves unavoidable, stop and report — licence agreements require a supervisor decision.

Task A5 — Load the target schema *1 hour*

Goal. Confirm the extracted target schema loads in your environment.

Steps. Copy `tpcdi_target_schema.sql` into `~/benchmark/tpcdi/`. Then create a script file. Type this command exactly — it writes a small Python program into a file called `load_target.py`:

```
cd ~/benchmark/tpcdi
pip install duckdb

cat > load_target.py << 'ENDOFFILE'
```

```
import duckdb, re

sql = open('tpcdi_target_schema.sql').read()
sql = re.sub(r'^\s*--.*', '', sql, flags=re.M) # remove comment lines

con = duckdb.connect('tpcdi.duckdb')
for statement in sql.split(';'):
    statement = statement.strip()
    if statement:
        con.execute(statement)

count = con.execute('SELECT count(*) FROM information_schema.tables').fetchone()[0]
print('tables created:', count)
ENDOFFILE

python3 load_target.py
```

Expected output: `tables created: 20`

If you get a different number, or an error, paste it verbatim into `FINDINGS.md`.

3 Benchmark 2: MIMIC-IV → OMOP

3.1 What this is

MIMIC is a database of real hospital intensive-care records from a Boston hospital, released for research. **OMOP CDM** ("Common Data Model") is a standard structure that medical researchers have agreed on, so that studies can run the same way on data from different hospitals.

Converting MIMIC into OMOP is a genuine, real-world migration. Somebody has already written the conversion, and published it. That makes it a promising example for us.

3.2 Two versions — use the right one

	MIT-LCP/mimic-omop	MIT-LCP/mimic-iv-demo-omop
Data version	MIMIC-III	MIMIC-IV, demo subset
Access	Requires credentialing: an online ethics course plus an application. Takes weeks.	**Open. No application needed.**
Language	PostgreSQL	SQL, run by a Python script
Use this one?	Not for now	**Yes — start here**

MIMIC-III access requires credentialing and a supervisor decision. The demo subset is open and sufficient to assess the benchmark's suitability.

3.3 What we already know (from reading documentation only)

Finding	Detail
It is staged	The conversion happens in named steps. Tables are prefixed by role: `src_` (a snapshot of the source), `lk_..._clean` (cleaned), `lk_..._concept` (codes mapped), `lk_..._mapped` (joined together), `cdm_` (the final target). This is useful — each step is separately inspectable.
It is many-to-one	In the MIMIC-III version, one target table is often built from several source tables. For example the `MEASUREMENT` table draws on five different source tables. This is the kind of complexity we want.
It uses a vocabulary	Medical codes in the source (e.g. `'44054006'`) must be translated into OMOP's numeric concept IDs (e.g. `201826`) by looking them up in a large reference table downloaded separately.

About the vocabulary. At first this looked like a problem, because the answer is not derivable from the source and target schemas alone. But if we *load the vocabulary table as part of the source database*, then looking up a code becomes an ordinary join against a source table — which is exactly the kind of transformation we are studying. So this is acceptable. Just be aware the vocabulary file is large.

3.4 MIMIC tasks

Task B1 — Clone and survey *2 hours*

```
cd ~/benchmark
mkdir -p mimic && cd mimic
git clone https://github.com/MIT-LCP/mimic-iv-demo-omop.git
cd mimic-iv-demo-omop
find . -type f -name "*.sql" | sort
find . -type f -name "*.md" | sort
```

Deliverable. In `FINDINGS.md`:

1. The complete list of `.sql` files, with their folder paths.
2. How many files are in each folder.
3. The contents of the main README, summarised in your own words in 5–10 lines: what does this repository claim to do, and what does it say you need to run it?

Task B2 — Find out where the data comes from *half a day*

Goal. Locate D.

Context. The repository holds the conversion code; the data is hosted separately on PhysioNet. The demo subset is openly downloadable.

Steps.

1. Read the repository README and find where it says the data comes from.
2. Go to https://physionet.org/content/mimic-iv-demo/ and check: can the files be downloaded without logging in? Look for a "Files" section at the bottom of the page.
3. If yes, download them into `~/benchmark/mimic/data/`.
4. List what you got and how big each file is: `ls -lh`.
5. Show the first 3 lines of two of the CSV files using `head -3`.

Deliverable. A table in `FINDINGS.md` listing every data file, its size, its number of columns, and its column names.

Also establish whether the conversion requires the OMOP vocabulary files from https://athena.ohdsi.org/, which need a free account. Report the requirement and the approximate size; the account decision sits with the supervisor.

Task B3 — Identify S, T, and the transformation *1 day*

Goal. Map the files in the repository onto our four pieces.

Steps. Go through the `.sql` files you listed in B1 and sort each one into a bucket:

Bucket	How to recognise it	Corresponds to
source structure	`CREATE TABLE` statements for MIMIC tables (names like `patients`, `admissions`)	S
target structure	`CREATE TABLE` statements for OMOP tables (names like `person`, `visit_occurrence`)	T
transformation	`INSERT INTO ... SELECT ...` statements	the reference program
helper	Anything else — config, cleanup, tests	—

Deliverable. A table in `FINDINGS.md`: file path, bucket, and a one-line description. Then state clearly which of the four pieces we now have and which we are still missing.

Task B4 — Classify the transformations *1 day. Most important MIMIC task.*

Goal. Apply the Task A3 classification to this benchmark.

Steps. Pick three target tables. Suggested: `person` (simplest), `visit_occurrence` (medium), `measurement` (most complex). Find the SQL that fills each, and classify every output column using the *same category list as Task A3*.

Add one extra category specific to this benchmark:

vocabulary lookup	The value comes from joining the large OMOP `concept` reference table to translate a medical code into a numeric ID.

Deliverable. `mimic_transformations.csv`, same columns as `tpcdi_transformations.csv`, plus counts per category.

Also answer: **for each of the three tables, how many different source tables does it read from?** This tells us how "many-to-one" the conversion is, which is a property we particularly want.

Task B5 — Try to run it *1–2 days. Attempt only after B1–B4.*

Goal. Get the conversion running end to end, so we have a working D and the converted output side by side.

Steps. Follow the repository's own instructions. The code targets a specific environment, so adaptation will be needed.

On failure, record in `FINDINGS.md`:

1. The exact command you ran.
2. The complete error message, copied and pasted, not summarised.
3. What you tried in response.

Optional. The classification in B4 is what decides whether we adopt this benchmark. Cap this task at two days; if it is still blocked, report the blocker and move on.

4 What to report back

4.1 Deliverables checklist

	File	Contents	From task
1	`FINDINGS.md`	Everything you ran and what happened, task by task	all
2	`tpcdi_transformations.csv`	Every target column classified	A3
3	`mimic_transformations.csv`	Every target column classified	B4
4	`tpcdi_source_schema.sql`	The source schema, if R1 provides it	A2
5	`sample_data/`	A few small sample data files from each benchmark	A4, B2

4.2 Questions the report must answer

For **each** of the two benchmarks, answer these explicitly. Answer each directly, with the evidence that supports it.

1. **Do we have all four pieces?** For S, D, T and the transformation: yes or no, and where each one is.
2. **How varied are the transformations?** Give the count per category. State the percentage that are plain copies.
3. **How many-to-one is it?** For your three sampled target tables, how many source tables does each one read?
4. **What blocks us?** Registration, licences, missing files, code that will not run — list each blocker and how long it would take to clear.

4.3 Interpretation of results

Outcome	What it means
Under 30% of columns are plain copies, and tables read from 2+ sources	Excellent. Use this benchmark.
30–70% plain copies	Usable. We will select the interesting subset of tables.
Over 70% plain copies	Too easy on its own. We would need to combine it with another source.
Blocked by registration or licensing	Report it. The supervisor decides.

4.4 Working notes

- **Registration and licensing decisions sit with the supervisor.** This covers tpc.org, PhysioNet credentialing, and Athena. Report what is required rather than signing up.
- **Error messages go into the report verbatim**, not paraphrased.
- **Negative findings are results.** "This repository does not contain the transformation SQL" resolves an open question and should be reported as such.
- **Every claim in this document about repository contents is an expectation drawn from README text, not a verified fact.** Some will be wrong. Establishing which is the point of tasks A1–A3 and B1–B4.

- **Flag blockers early.** A day lost to a setup problem is worth a message.

4.5 Suggested order and timing

Day	Task	What	Hours
1	Setup	Install WSL, Ubuntu, Python, git, VS Code	3
1	A1	Clone TPC-DI repos, list contents	2
1	B1	Clone MIMIC repo, list contents	2
2	A2	Read the TPC-DI source schema file	2
2	A5	Load the target schema into DuckDB	1
2	B2	Find and download the MIMIC demo data	4
3	A3	**Classify TPC-DI transformations**	7
4	B3	Sort MIMIC files into buckets	4
4	A4	Investigate the TPC-DI data generator	3
5	B4	**Classify MIMIC transformations**	7
6–7	B5	Try to run the MIMIC conversion (optional)	10
7	—	Write up and present findings	3

A3 and B4 are the deciding tasks; the remainder is preparation for them.